Schema Refinement and Normal Forms

Ioana Manolescu
INRIA and Ecole Polytechnique



Slides Courtesy of R. Ramakrishnan, J. Gehrke and Y. Diao

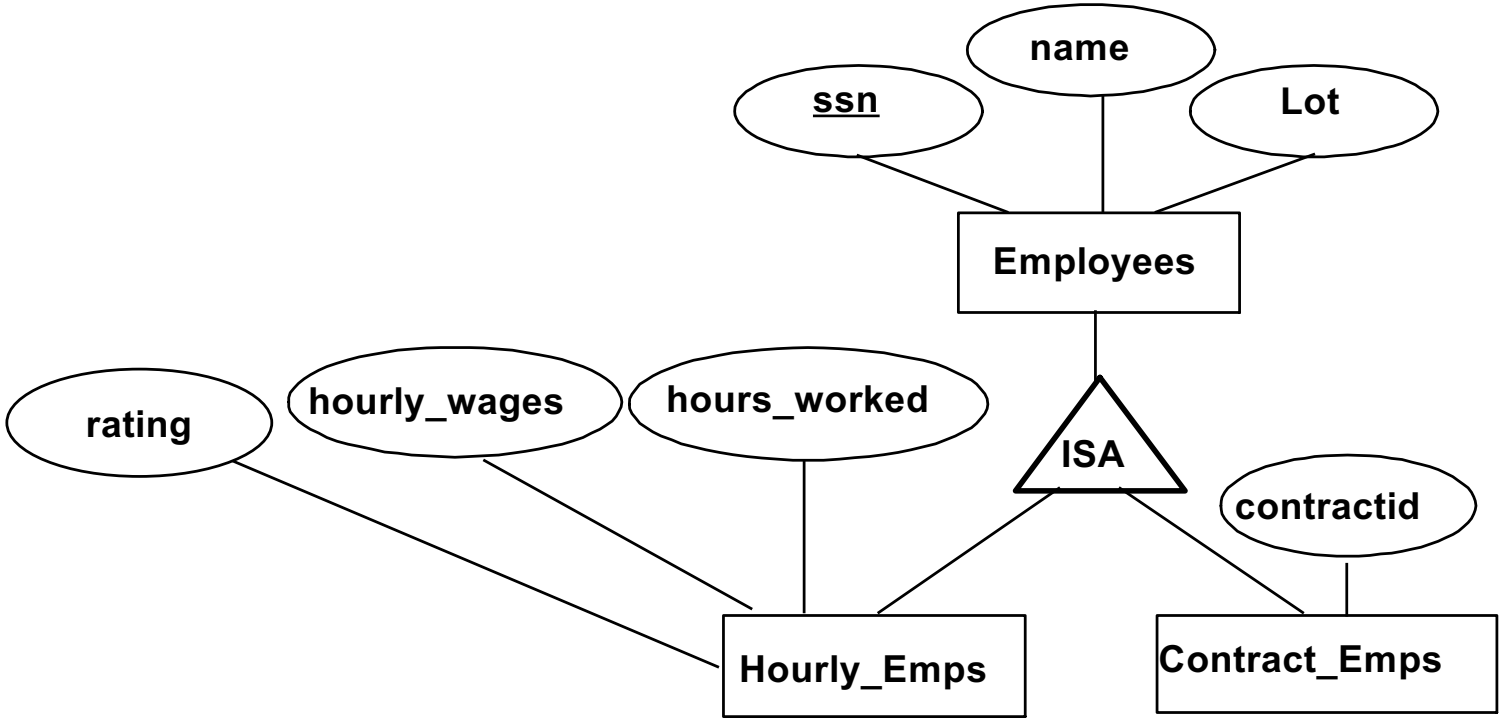
Problems in a relational schema

What can go wrong in a relational schema

- We have seen:
  - Requirement analysis leads to ER model and then relational schema
  - Query evaluation, evaluation cost, performance problems
    - Mostly due to joins!
- Today:

1. **Problems** which may occur in a relation
2. How to **model and analyze** the problems?
3. When is each kind of problem **avoided**?
4. How to **fix** the schema when a relation has problems?

A relation, its attributes, and their problems



- Consider relation obtained from **Hourly_Emps**:
  - Hourly_Emps(*ssn, name, lot, rating, hrly_wages, hrs_worked*)
    - *lot* is the parking lot where each employee has a place
  - Denote the schema by listing all its attributes: SNLRWH

Anomalies of a relation

S	N	L	R	W	H
123-22-3666	Attishoo	48	8	10	40
231-31-5368	Smiley	22	8	10	30
131-24-3650	Smethurst	35	5	7	30
434-26-3751	Guldu	35	5	7	32
612-67-4134	Madayan	35	8	10	40

- Suppose rating (R) determines hourly wages (W)
  - *Redundant storage:* we store (R=8, W=10) three times, (R=5, W=7) twice
    - Waste of space
    - *Risk of introducing inconsistencies:* one may update e.g., W just in the first tuple by mistake, compromising the database rule that R determines W

Anomalies of a relation

S	N	L	R	W	H
123-22-3666	Attishoo	48	8	10	40
231-31-5368	Smiley	22	8	10	30
131-24-3650	Smethurst	35	5	7	30
434-26-3751	Guldu	35	5	7	32
612-67-4134	Madayan	35	8	10	40

- Rating (R) determines hourly wages (W):
  - *Insertion anomalies:*
    - Cannot insert an employee without knowing the hourly wage for his rating
      - We could insert it with a NULL wage
    - Cannot insert the wage for a given rating without an employee
      - Cannot insert a tuple with a null S (primary key!)

Anomalies of a relation

S	N	L	R	W	H
123-22-3666	Attishoo	48	8	10	40
231-31-5368	Smiley	22	8	10	30
131-24-3650	Smethurst	35	5	7	30
434-26-3751	Guldu	35	5	7	32
612-67-4134	Madayan	35	8	10	40

- Rating (R) determines hourly wages (W):
  - *Deletion anomaly*: If we delete all employees with a given rating, we lose the information about the wages associated to that rating

Will Two Smaller Tables be Better?

S	N	L	R	W	H
123-22-3666	Attishoo	48	8	10	40
231-31-5368	Smiley	22	8	10	30
131-24-3650	Smethurst	35	5	7	30
434-26-3751	Guldu	35	5	7	32
612-67-4134	Madayan	35	8	10	40

Hourly_Emps2

S	N	L	R	H
123-22-3666	Attishoo	48	8	40
231-31-5368	Smiley	22	8	30
131-24-3650	Smethurst	35	5	30
434-26-3751	Guldu	35	5	32
612-67-4134	Madayan	35	8	40

Wages

R	W
8	10
5	7

No redundancy
No anomalies

Reasoning about the problems: functional dependencies

The Evils of Redundancy

- Redundant storage causes several operation anomalies:
  - Insert/delete/update anomalies
- *Functional dependencies*, a new type of integrity constraint, can be used to identify schemas with such problems.
  - IC's we have seen: *attribute constraints, key constraints, foreign key constraints, general constraints*
  - A new type of integrity constraint: *functional dependencies*

Functional Dependencies (FDs)

- A *functional dependency* X → Y holds over relation R if:
  - X and Y are two sets of attributes of R;
  - ∀ allowable instance *r* of R:

Notation:
X, Y **attribute sets**

$$t1 \in r, t2 \in r, \pi_X(t1) = \pi_X(t2) \text{ implies } \pi_Y(t1) = \pi_Y(t2)$$

- An FD holds for *all* allowable instances of a schema.

- Key constraint is a special form of FD:
  - K is a candidate key for R means that K → R.
  - In a *functional dependency* K → R, K does not have to be minimal; in a *key constraint*, yes. (Otherwise K is a superkey.)

FDs in the Hourly_Emps Example

- **Hourly_Emps**(*ssn, name, lot, rating, hourly_wages, hours_worked*)
  - Denoted by SNLRWH

- Some FDs on Hourly_Emps:
  - *ssn* is the key: S → SNLRWH
  - *rating* determines *hrly_wages*: R → W

Reasoning About FDs

- Given some FDs, we can usually infer additional FDs:
  - *ssn* → *did*, *did* → *lot* implies *ssn* → *lot*
- An FD *f* is *implied by* a set of FDs *F*, if *f* holds for every reln instance that satisfies all FDs in *F*.
  - ***F*⁺ = *Closure of F*** is the set of all FDs that are implied by *F*.

- **Armstrong's Axioms** (X, Y, Z are sets of attributes):
  - *Reflexivity*: If $X \subseteq Y$, then $Y \rightarrow X$
  - *Augmentation*: If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z
    - Caution: $XZ \rightarrow YZ$ does **not** entail $X \rightarrow Y$
  - *Transitivity*: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Reasoning About FDs (Contd.)

- Couple of additional rules (that follow from AA):
  - *Union*: If X → Y and X → Z, then X → YZ
  - *Decomposition*: If X → YZ, then X → Y and X → Z
- AA are *sound* and *complete* inference rules for FDs
  - *Soundness*: when applied to a set *F* of FDs, the axioms generate only FDs in F^+.
  - *Completeness*: repeated application of these axioms will generate all FDs in F^+.
- We will often work only on dependencies with 1 attribute on the right hand side
  - Union allows to combine these into others that have more attributes on the right

Reasoning About FDs (Contd.)

- Computing the closure F^+ can be expensive:
  - Compute for *all* FD's.
  - In the worst case, size of closure is exponential in number of attributes

- Typically, we just want to check if *a given* FD $X \rightarrow Y$ is in F^+. An efficient check:
  1. Compute *attribute closure* of X (denoted X^+) w.r.t. F, that is: the *largest* attribute set Z such that $X \rightarrow Z$ is in F^+.
  2. Check if $Y \subseteq X^+$.

Checking if a functional dependency holds

- What if we just want to check if *a given* FD $X \rightarrow Y$ is in F^+?
- Simple algorithm for *attribute closure* X^+:
  - $X^+ := \{X\}$
  - DO if there is U → V in F, s.t. $U \subseteq X^+$,

    then $X^+ = X^+ \cup V$

    UNTIL no change
- Check if *a given* FD $X \rightarrow Y$ is in F^+:
  - Simply check if $Y \subseteq X^+$.
- Does F = {A → B, B → C, C D → E } imply A → E?
  - Is A → E in the closure F^+?
  - Equivalently, is E in A^+?

Avoiding the problems: normal forms

Normal Forms

- Role of FDs in detecting redundancy: R(A,B,C)
  - *No FDs hold*: No redundancy here.
  - *Given A → B*: Two tuples have the same A value will have the same B value!
- This entails: redundancy, update/insertion/deletion anomalies
- ***Normal forms***: If a relation does not have **certain kinds of FDs**, certain redundancy-related problems are known not to occur.

The Main Two Normal Forms

- 3rd normal form (3NF): for any dependency **X→A** **(A is an attribute)**, one of the following is true:
  - A in X (trivial dependency), or
  - X is a superkey, or
  - A is part of a key.
- Boyce-Codd normal form (BCNF): for any dependency **X→A**, one of the following is true:
  - A in X (trivial dependency), or
  - X is a superkey.

Notation:
in X→A,
A **is <u>one</u> attribute**

BCNF implies 3NF; BCNF more restrictive

Boyce-Codd Normal Form (BCNF)

- A relation R is in Boyce-Codd normal form *iff* for any FD X→A holding on R, either:
  - A is included in X (called trivial dependency), or
  - X is a superkey (recall: keys are minimal superkeys)

- **Equivalent definition:**
  Transform F^+ into an equivalent set of dependencies by transforming every X→Y, where Y={A_1, A_2, ..., A_n}
  into: X→A_1, X→A_2, ... , X→A_n

  R is in BCNF iff for any FD X→ A (A is an attribute):

  1. A ∈ X (trivial FD), or
  2. X is a *superkey* (i.e., contains a key) for R.

- **In BCNF, the only non-trivial FDs are key constraints!**

Boyce-Codd Normal Form (contd.)

- Can we infer the '?' value ?
  - If yes, then X → A
  - If X → A and X is not a superkey, then the relation is not in BCNF
    - X is clearly not a superkey if y1 ≠ y2
    - Thus, if X→A, the relation is not in BCNF

X	Y	A
x	y_1	a
x	y_2	?

- Intuition: in a BCNF relation, *every field of every tuple stores information that can't be inferred using FD's and the other fields.*
  - *"Nothing can be inferred" = "nothing is redundant"*
  - If R has a superkey X, we have X→R, but (by definition of a relation) each X value tuple only occurs once
  - It it has no superkey, nothing can be inferred
- What BCNF achieves is:
  - *No redundancy can be detected using FDs!*

Third Normal Form (3NF)

- Relation R with FDs *F* is in **3NF** if $\forall$ X → A in F^+:
  1. A $\in$ X (called a *trivial* FD), or
  2. X is a *superkey* for R, or
  3. A is part of some *key* for R (*minimality* of a key is crucial in the third condition).

- If R is in BCNF, obviously in 3NF.

Third Normal Form (contd.)

- If R is in 3NF, some *redundancy is possible*!
  - **Reserves**(Sailor, Boat, Date, Credit_card) with
    S → C, C → S
  - Keys are SBD and CBD.
  - It is in 3NF.
  - But for each reservation of sailor S, same (S, C) is stored.

More on BCNF and 3NF

To check if a relation R is in BCNF or 3NF, need to compute *all* the keys of R.

To enforce FDs in BCNF or 3NF, declare key constraints and checks in CREATE TABLE.

- **Reserves**(Sailor, Boat, Date, Credit_card)

- 3NF is a weaker normal form than BCNF *but:*
  - Always possible to have *lossless-join, dependency-preserving* decomposition of R into 3NF (seen next).
  - This is not true for BCNF!

Solving the problems which may arise: schema refinement

Schema Refinement

- Main refinement technique: *decomposing* a relation into multiple smaller ones
- Decomposition should be used judiciously:
  - Is there **reason** to decompose a relation?

    Theory on *normal forms*.
  - What **problems** (if any) does the decomposition cause?
    Properties of decomposition include *lossless-join* and *dependency-preserving*.
  - Decomposition can cause **performance** problems.

    E.g. a previous selection now requires a join!

Decomposing a Relation Scheme

- A *decomposition* of R **breaks** R into two or more relations s.t.
  - Each new relation contains a subset of the attributes of R.
  - Every attribute of R appears in at least one new relation.

- Decompositions should be used only when:
  - R has **redundancy related problems** (not in BCNF or 3NF),
  - **We can afford the joins** in queries later.

Problems with Decompositions

- Three potential problems to consider:
  - **Some queries become more expensive.**
    - e.g., How much did sailor Joe earn? (salary = W*H)
    - Performance problem
  - **Given instances of the decomposed relations, we may not be able to reconstruct the corresponding instance of the original relation**
    - Correctness problem – inacceptable!
  - **Checking some dependencies may require joining the instances of the decomposed relations.**
    - Performance problem and/or impossible to do
    - Typically avoided

- *Tradeoff*: Must consider these issues vs. redundancy.

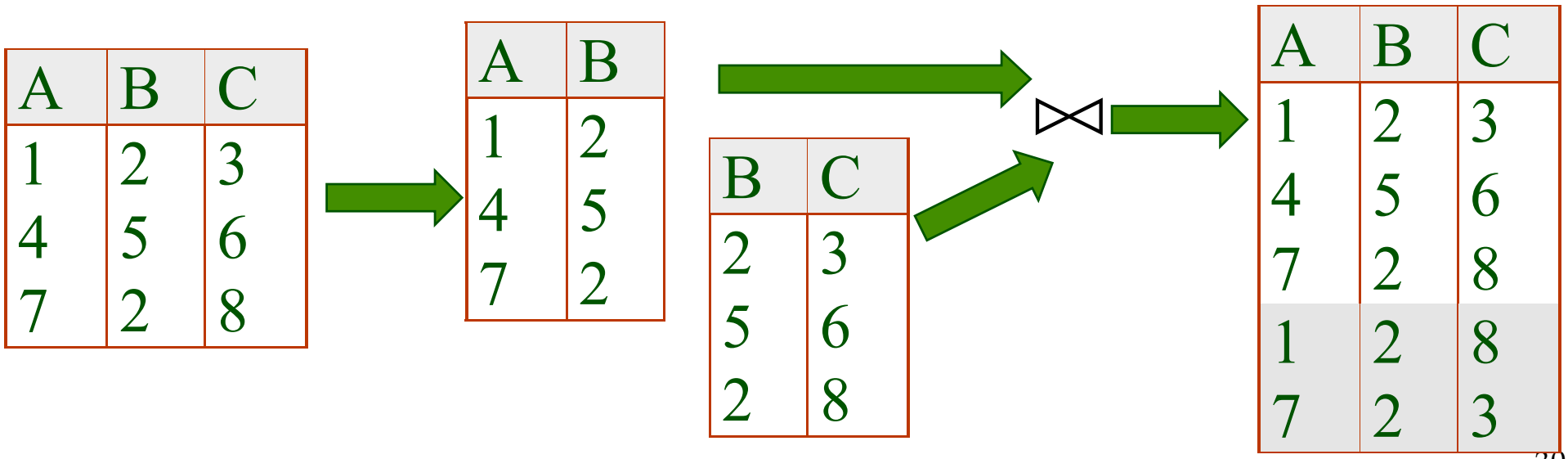
Example Decomposition

- Hourly_Emps (SNLRWH)
  - FDs: S → SNLRWH and R → W.
  - **R → W** violates 3NF.
  - And it causes repeated (R,W) storage.

- To fix this, create a relation RW, remove W from the main schema. (SNLRWH) becomes: (SNLRH) and (RW).

Lossless Join Decompositions

- Decomposition of R into R1 and R2 is *lossless-join* w.r.t. a set of FDs F if ∀ instance *r* that satisfies F:
  - $r = \pi_{R1}(r) \bowtie \pi_{R2}(r)$
- It is always true that $r \subseteq \pi_{R1}(r) \bowtie \pi_{R2}(r)$.
  - A bad decomposition can cause $r \subset \pi_{R1}(r) \bowtie \pi_{R2}(r)$.



A Simple Test for Lossless Join

- Theorem:
  Decomposition of R into R1 and R2 is *lossless-join* wrt F **iff** F^+ contains:
  - **R1 ∩ R2 → R1** or **R1 ∩ R2 → R2**
  - The intersection of R1 and R2 is a (super) key of one of them.

> *If* ***R1 ∩ R2 → R1, then:***
> *Every R2 tuple, which includes the attrs. R1 ∩ R2,*
> *exists in R with only one value for R1 attrs.*
> *Thus, we can get back exactly R by extending R2 with the R1 'function'.*

- How to apply this result?
  If **U → V** holds over R and violates a BCNF definition,
  the decomposition into **UV** and **R − V** is lossless-join.

Dependency Preserving Decomposition

- **Contracts**(contractid, supplierid, projectid, deptid, partid, qty, value), CSJDPQV, with FDs:
  - **C** is key.
  - **JP → C**: a project buys a given part using a single contract.
  - **SD → P**: a department buys at most one part from a supplier.
- What are the keys? Which normal form is it in?
  - C, JP, SDJ. 3NF.
- Lossless-join BCNF decomposition: CSJDQV, SDP
  - Problem: Checking **JP → C** requires an assertion (using join)!

The decomposition is not *dependency-preserving*.

Dependency Preserving Decomposition

- The *projection* of a FD set F onto a decomposed relation R1, denoted F_{R1}, is the set all FDs in F^+ of the form U→V such that both U and V are in R1. *"FDs local to R1"*
- Decomposition of R into R1, R2 is *dependency preserving* if $(F_{R1} \text{ UNION } F_{R2})^+ = F^+$
- Important to consider F^+ (**not** F!) in this definition:
  - ABC, A → B, B → C, C → A, decomposed into AB and BC.
  - Is this dependency preserving? Is C → A preserved?
- A decomposition can be:
  - lossless, but not dependency preserving
  - dependency preserving, but not lossless

Normalization

- The process of taking **a relation and its set of FDs**, and turning it into **a set of BCNF or 3NF relations**
- We will see:

1. Any relation has a lossless-join BCNF decomposition
   - In some cases, none of these decompositions preserves dependencies!
2. Any relation has a 3NF *dependency-preserving*, lossless-join decomposition
   - This is the interest of 3NF as a "weak form of BCNF"

Decomposition into BCNF

- Relation R with FDs F. If **X → Y** violates BCNF, decompose R into **R1=R − Y** and **R2=XY**.
  - For each Ri, compute F_{Ri} and check *if Ri is in BCNF*.
  - If not, pick a FD violating BCNF and keep decomposing Ri.

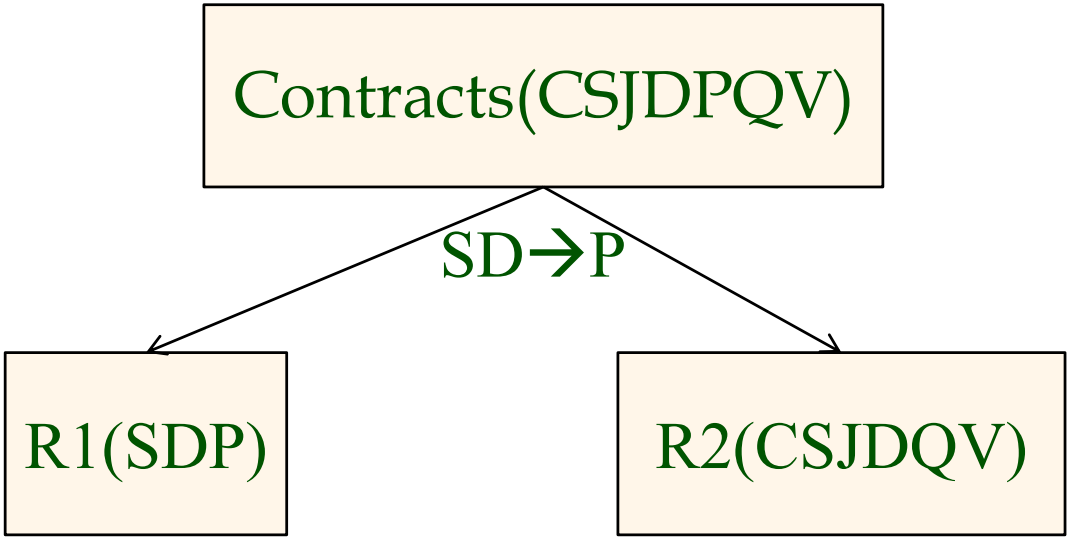
- Repeated application of this process yields a *lossless join* decomposition into *BCNF* relations.

BCNF Decomposition Example

- Contracts(CSJDPQV), key C, JP → C, SD → P, J → S.
  1. *Keys.* C; JP because JP → C;
     DJ because

     J→S, thus DJ→SD and SD→P, thus DJ→P, DJ→JP and JP is a key.
  2. *Normal form.* Not in BCNF: SD → P and J → S violate BCNF

     SD is not a key, J is not a key
  3. *Decomposition.* To deal with SD → P, decompose into SDP, CSJDQV.



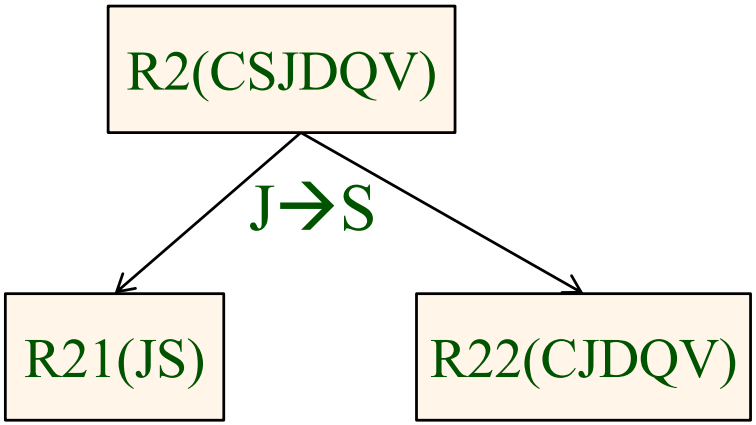
BCNF Decomposition Example

- Contracts(CSJDPQV), key C, JP → C, SD → P, J → S.
  - **SDP** is in BCNF. But CSJDQV is not because:

  1. FDs and keys. Projection of FDs from Contracts: keys C and DJ, J → S.

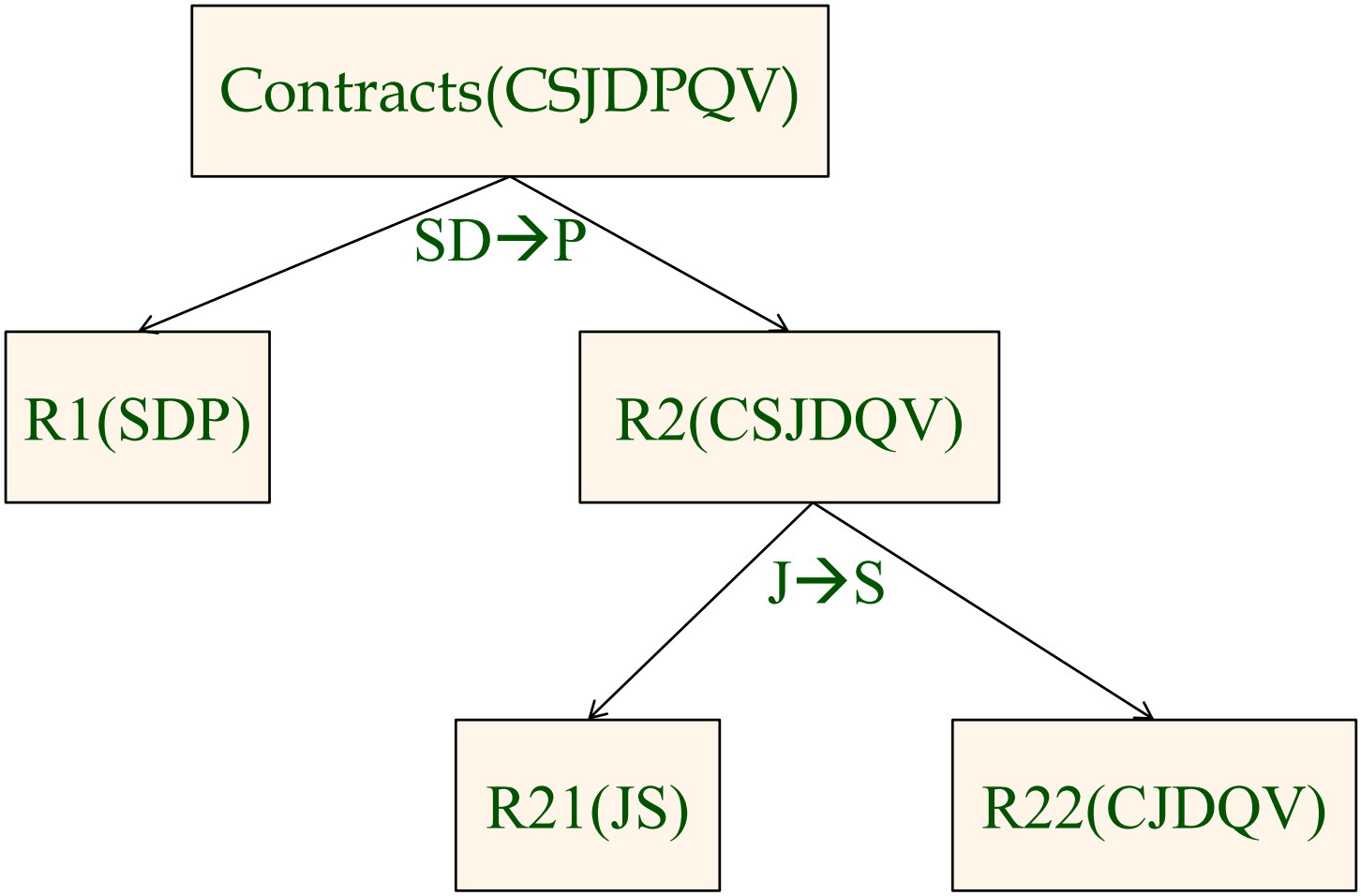
  2. Normal form. Not BCNF; J → S violates BCNF.

  3. Decomposition. For J → S, decompose CSJDQV into JS and CJDQV.



  - **JS** is in BCNF. So is **CJDQV**.
- If several FDs violate BCNF, the order of "dealing with" them may lead to different decompositions.

BCNF decomposition



BCNF and Dependency Preservation

- Is a *lossless-join BCNF* decomposition *dependency-preserving*?
  - CSJDQV with JP → C, SD → P and J → S.
  - CSJDQV decomposed into SDP, JS and CJDQV
  - What about JP → C?

BCNF and Dependency Preservation

- Is a *lossless-join BCNF* decomposition *dependency-preserving*?
  - CSJDQV with JP → C, SD → P and J → S.
  - CSJDQV decomposed into SDP, JS and CJDQV
  - What about JP → C?
    - Adding JPC as a new relation to preserve JP → C introduces *redundancy across relations.*
    - If we also have J→C, JPC is not in BCNF, SD→P cannot be preserved, etc.
- In general, there may not exist a *lossless join, dependency-preserving* decomposition into *BCNF.*
  - But there is always a *lossless join, dependency-preserving* decomposition into *3NF.*

Decomposition into 3NF

- The algorithm for lossless join decomposition into BCNF can be used to obtain a lossless join decomposition into 3NF (typically, can stop earlier).
- Idea to ensure dependency preservation: *If $X \rightarrow Y$ is not preserved,* ***add relation XY.***
  - Problem: XY may violate 3NF
  - E.g., suppose $AB \rightarrow C$ is lost in decomposition. Add ABC to 'preserve' $AB \rightarrow C$. What if we also have $A \rightarrow B$?
- Refinement: Instead of the given set of FDs F, use a *minimal cover for F* (minimal FD set G s.t. $G^+ = F^+$).

Decomposition into 3NF, refined

- Step 1: Given a set *F* of FDs, compute its *minimal cover G* (see next)
- Step 2: Use *G* to create a **lossless-join decomposition** of R into R1, …, Rn.
- Step 3: Identify the dependencies in *G* that are **not preserved**. For each such FD X→A, add a new relation XA.
  - *XA is guaranteed to be 3NF.*
    - *X is a key;*
    - *Any other G dependency within XA has an X attr at right.*
- This algorithm produces a *lossless-join, dependency-preserving* decomposition into *3NF.*

Minimal Cover for a Set of FDs

- *Minimal cover* G for a set of FDs F:
  - Closure of F = closure of G.
  - Right hand side of each FD in G is a single attribute.
  - Removing any FD from G changes the closure.
  - Removing any attribute from the left of any FD changes the closure.
- Intuitively, *every FD in G is needed*, and "*as small as possible*" in order to get the same closure as F.
- E.g., A → B, ABCD → E, EF → GH, ACDF → EG has the following minimal cover:
  - A → B, ACD → E, EF → G and EF → H

3NF decomposition with the minimal cover leads to Lossless-Join, Dependency-Preserving decomposition.

Algorithm for obtaining MC of F

1. Put all the FDs in **standard form**: 1 attribute at the right
2. **Minimize** the left side of each FD
    1. Remove any attribute that can be removed while preserving the closure. This involves checking all FDs!
3. **Delete redundant** FD
    1. Remove any FD that can be removed while preserving the closure

Guidelines on Schema Refinement

- If a relation **is in BCNF**, it is free of redundancies that can be detected using FDs.
- If a relation **is not in BCNF**, try to decompose it into a collection of BCNF relations.
  - Must consider whether all FDs are preserved.
  - If a dependency preserving decomposition into BCNF is not possible, consider decomposition into 3NF.
  - Decompositions should be carried out while keeping *performance requirements* in mind.

Questions

